

6. PROJECT DEVELOPMENT PHASE

Project title: Multi-Line Insurance Policy and Claims Management System

Team Members:

Dhanasri G - Team Lead

Abinaya S - Team Member

Jamunadevi R - Team Member

Miruthulasri G - Team Member

Kanagalakshmi G - Team Member

Phase 1: Requirement Analysis & Planning

Objective

The main objective of this phase is to understand the requirements of the Multi-Line Insurance Policy and Claims Management System and plan the Salesforce implementation.

Insurance companies manage different types of insurance policies such as Life, Health, Motor and Property Insurance. Managing customer details, policy information, premium payments and claims through separate systems can make the process difficult and time-consuming.

The proposed system provides a centralized Salesforce platform where customer information, insurance policies, claims and payment details can be managed efficiently. The system also provides AI-based assistance through AgentForce AI for policy and claims-related queries.

Key Activities

- Collect requirements from customers, insurance agents, claims officers and administrators.
- Identify the major users of the system such as Customer, Insurance Agent, Claims Officer and Admin.
- Define the scope and objectives of the project.
- Identify insurance policy creation, management and renewal requirements.
- Identify claim registration, verification, processing and status tracking requirements.
- Design the data model using Customer, Insurance Product, Policy, Claim and Payment objects.
- Identify the Salesforce tools required for development.
- Plan automation using Salesforce Flows.
- Plan AI-based assistance using AgentForce AI and Prompt Builder.
- Define security requirements using Profiles and Permission Sets.
- Plan reports and dashboards for monitoring policies, claims and payments.

Phase 2: Salesforce Development – Backend & Configurations

This phase focuses on developing the backend structure and configuring the Salesforce environment for the Multi-Line Insurance Policy and Claims Management System. The required Salesforce objects, fields, relationships, tabs, Flows and AI components are configured to support insurance policy and claims management. This follows the same structure as the Salesforce Development phase in your reference document. Project Development phase.pdf

Key Activities

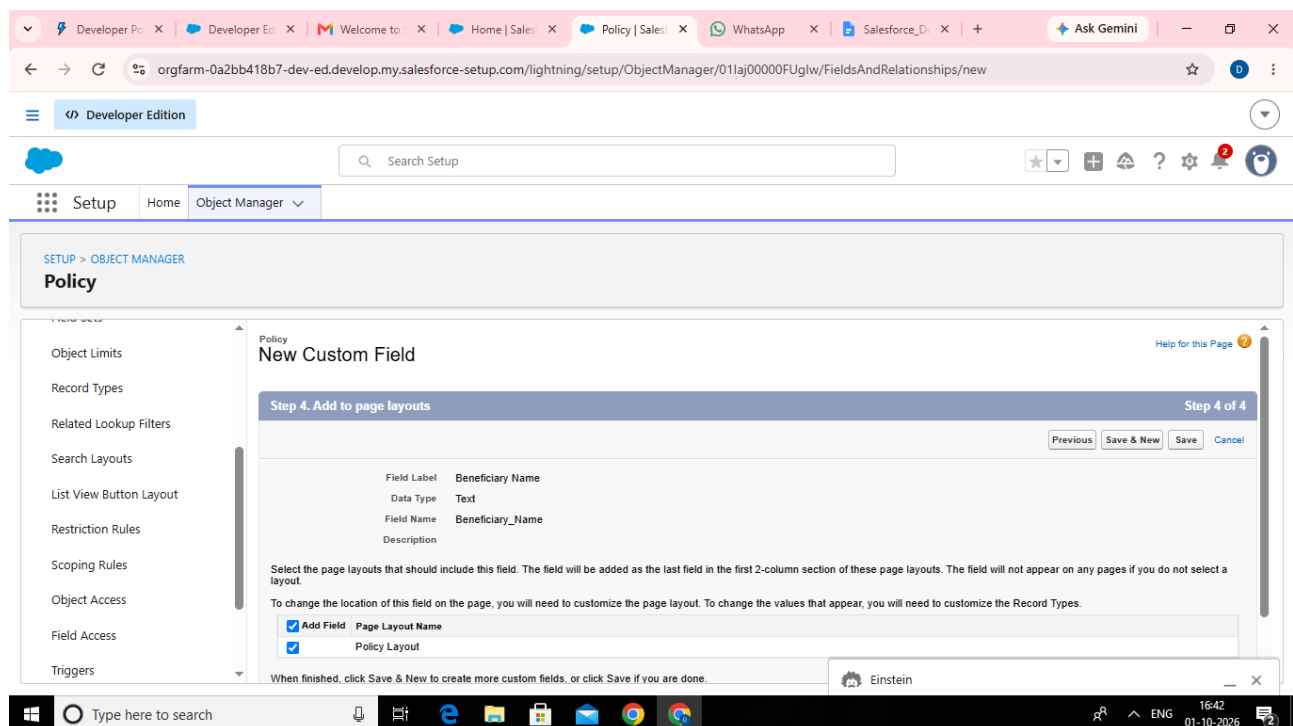
- Create and configure the Salesforce Developer environment.
- Create the required custom objects such as Customer, Insurance Product, Policy, Claim and Payment.

- Create relationships between the objects.
- Create Salesforce tabs for the project modules.
- Create fields for storing customer, policy, claim and payment information.
- Configure Salesforce Flows for process automation.
- Configure Prompt Builder for insurance policy and claims-related information retrieval.
- Configure the required backend components for policy and claims management.
- Configure the system to support multiple insurance lines such as Life, Health, Motor and Property Insurance.

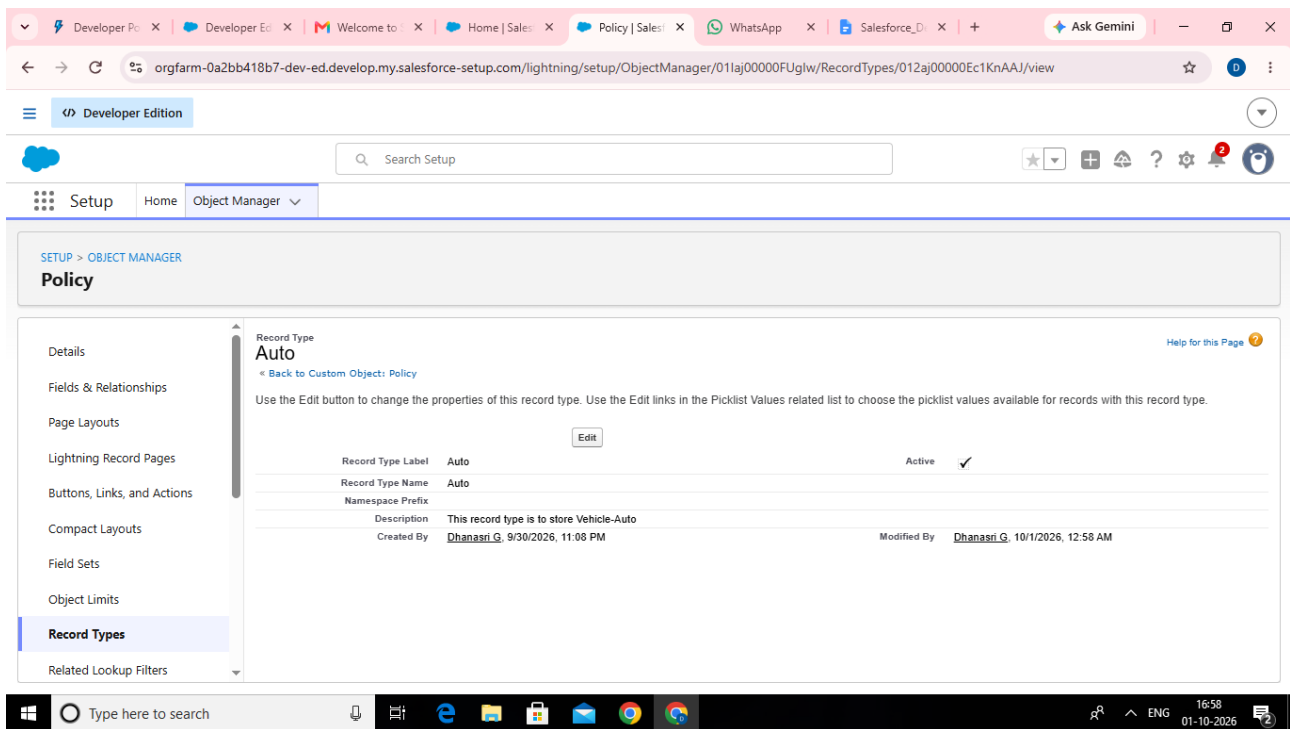
Milestone 1 – Salesforce Account

Create and configure the Salesforce Developer environment for developing the insurance policy and claims management system.

1. Create Policy custom object:

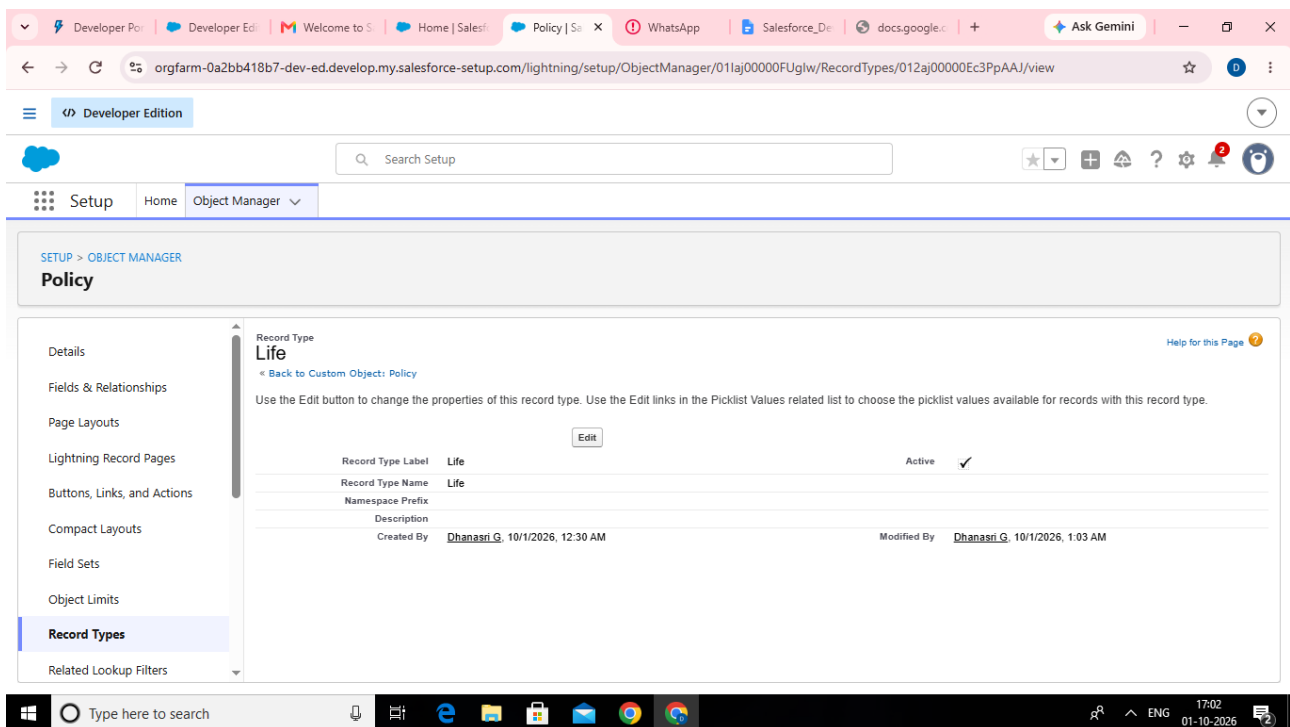


2. Define Record Types on Policy Object:



This screenshot shows the Salesforce Setup interface for defining a Record Type on the Policy object. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types (selected), and Related Lookup Filters. The main content area displays the 'Record Type' configuration for 'Auto'. It includes an 'Edit' button and a table with the following details:

Record Type Label	Auto	Active	✓
Record Type Name	Auto		
Namespace Prefix			
Description	This record type is to store Vehicle-Auto		
Created By	Dhanasri G. 9/30/2026, 11:08 PM	Modified By	Dhanasri G. 10/1/2026, 12:58 AM



This screenshot shows the Salesforce Setup interface for defining a Record Type on the Policy object. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types (selected), and Related Lookup Filters. The main content area displays the 'Record Type' configuration for 'Life'. It includes an 'Edit' button and a table with the following details:

Record Type Label	Life	Active	✓
Record Type Name <th>Life</th> <td></td> <td></td>	Life		
Namespace Prefix			
Description			
Created By	Dhanasri G. 10/1/2026, 12:30 AM	Modified By	Dhanasri G. 10/1/2026, 1:03 AM

Developer Por | Developer Edi | Welcome to S | Home | Salesf | Policy | Se x | WhatsApp | Salesforce_De | docs.google.c | + | Ask Gemini

orgfarm-0a2bb418b7-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/011aj00000FUGlw/RecordTypes/012aj00000Ec3MbAAJ/view

Developer Edition

Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Record Type Property

« Back to Custom Object: Policy

Use the Edit button to change the properties of this record type. Use the Edit links in the Picklist Values related list to choose the picklist values available for records with this record type.

Edit

Record Type Label	Property	Active	✓
Record Type Name	Property		
Namespace Prefix			
Description			
Created By	Dhanasri G. 10/1/2026, 12:28 AM	Modified By	Dhanasri G. 10/1/2026, 1:03 AM

Type here to search

ENG 17:03 01-10-2026

3. Create Field Sets for policy specific data:

Developer Por | Developer Edi | Welcome to S | Home | Salesf | Policy | Se x | WhatsApp | Salesforce_De | Multi-Line Ins | + | Ask Gemini

orgfarm-0a2bb418b7-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/011aj00000FUGlw/FieldSets/01Xaj00000B8WqnGAF/view

Developer Edition

Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Object Access

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Model Year	Policy Term Months	Square Footage
Created By	Last Activity Date	Owner	Premium	System Modstamp
Created Date	Last Modified By	Policy Number	Record ID	VIN
Customer	Last Modified Date	Policy Start Date	Record Type	Year Built

Available for the Field Set 1

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set 1

VIN

Model Year

Type here to search

ENG 17:09 01-10-2026

4. Property FieldSet Requires:

The screenshot shows the Salesforce Setup interface in Developer Edition. The breadcrumb trail is **SETUP > OBJECT MANAGER > Policy**. The left sidebar lists various configuration options, with **Field Sets** selected. The main content area displays the **Field Set Properties** for a field set named **Policy**. A table lists fields available for the field set, including Beneficiary Name, Deleted, Model Year, Policy Term Months, Square Footage, Created By, Last Activity Date, Owner, Premium, System Modstamp, Created Date, Last Modified By, Policy Number, Record ID, VIN, Customer, Last Modified Date, Policy Start Date, Record Type, and Year Built. Below this, the **Available for the Field Set** section shows a list of fields to be added to the field set, including Square Footage and Year Built.

Beneficiary Name	Deleted	Model Year	Policy Term Months	Square Footage
Created By	Last Activity Date	Owner	Premium	System Modstamp
Created Date	Last Modified By	Policy Number	Record ID	VIN
Customer	Last Modified Date	Policy Start Date	Record Type	Year Built

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

Square Footage
Year Built

5. Life FieldSet Requires:

The screenshot shows the Salesforce Setup interface in Developer Edition. The breadcrumb trail is **SETUP > OBJECT MANAGER > Policy**. The left sidebar lists various configuration options, with **Field Sets** selected. The main content area displays the **Field Set Properties** for a field set named **Policy**. A table lists fields available for the field set, including Beneficiary Name, Deleted, Model Year, Policy Term Months, Square Footage, Created By, Last Activity Date, Owner, Premium, System Modstamp, Created Date, Last Modified By, Policy Number, Record ID, VIN, Customer, Last Modified Date, Policy Start Date, Record Type, and Year Built. Below this, the **Available for the Field Set** section shows a list of fields to be added to the field set, including Beneficiary Name and Policy Term Months.

Beneficiary Name	Deleted	Model Year	Policy Term Months	Square Footage
Created By	Last Activity Date	Owner	Premium	System Modstamp
Created Date	Last Modified By	Policy Number	Record ID	VIN
Customer	Last Modified Date	Policy Start Date	Record Type	Year Built

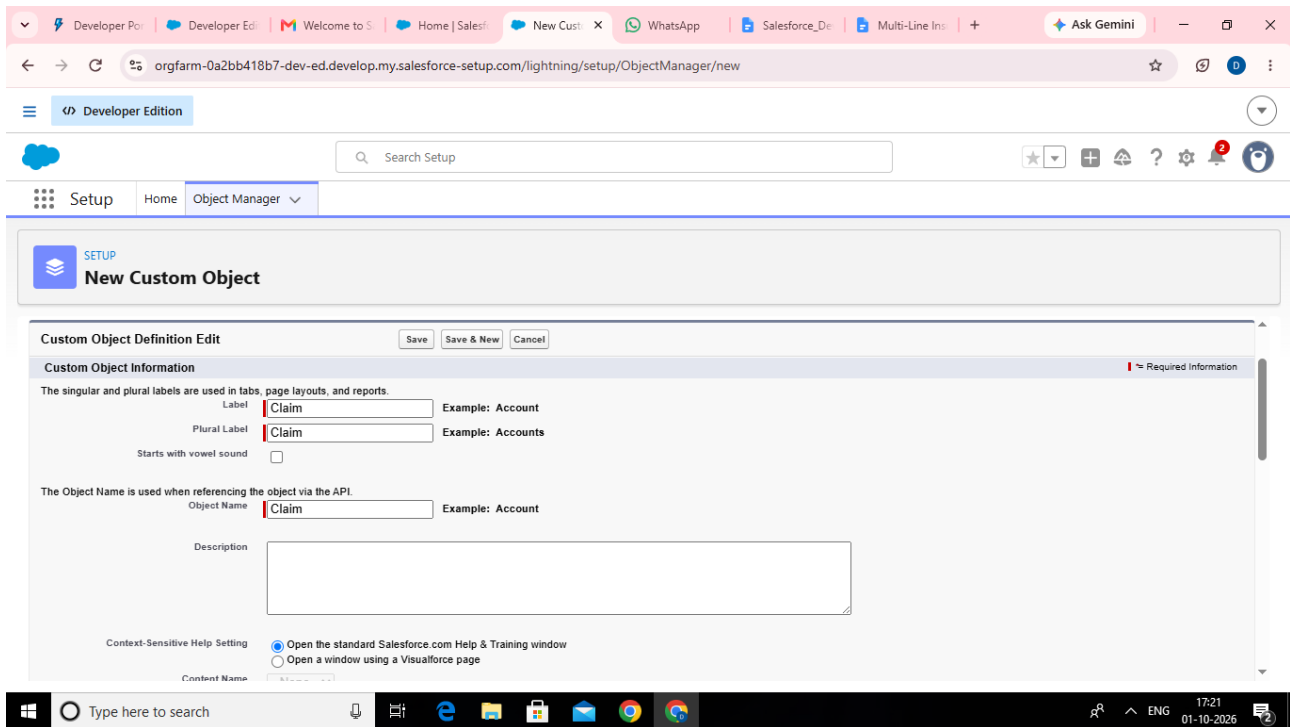
Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

Beneficiary Name
Policy Term Months

6. Create Claim Custom Object:



The screenshot shows the Salesforce Setup interface in Developer Edition. The 'Object Manager' tab is selected, and the 'New Custom Object' wizard is open. The 'Custom Object Definition Edit' window is displayed, showing the 'Custom Object Information' section. The 'Label' field is set to 'Claim', and the 'Plural Label' is also 'Claim'. The 'Object Name' is 'Claim', and the 'Description' field is empty. The 'Context-Sensitive Help Setting' is set to 'Open the standard Salesforce.com Help & Training window'.

Custom Object Definition Edit

Custom Object Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label: Example: Account

Plural Label: Example: Accounts

Starts with vowel sound: ☐

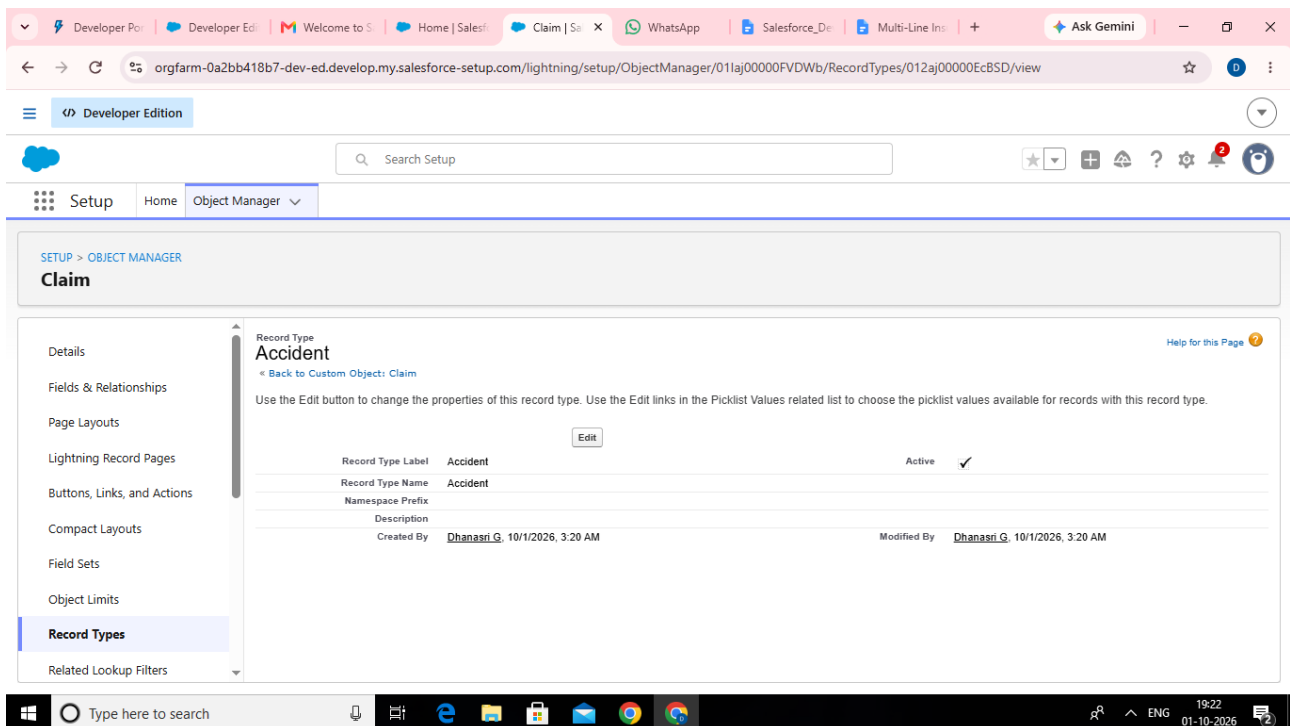
The Object Name is used when referencing the object via the API.

Object Name: Example: Account

Description:

Context-Sensitive Help Setting: ☒ Open the standard Salesforce.com Help & Training window ☐ Open a window using a Visualforce page

7. Define Record Types on Claim Object:



The screenshot shows the Salesforce Setup interface in Developer Edition. The 'Object Manager' tab is selected, and the 'Claim' object is chosen. The 'Record Types' section is active, showing the 'Accident' record type. The 'Record Type Label' is 'Accident', and the 'Record Type Name' is 'Accident'. The 'Active' checkbox is checked. The 'Created By' is 'Dhanasri G.' and the 'Modified By' is 'Dhanasri G.'.

SETUP > OBJECT MANAGER

Claim

Record Type

Accident

« Back to Custom Object: Claim

Use the Edit button to change the properties of this record type. Use the Edit links in the Picklist Values related list to choose the picklist values available for records with this record type.

Record Type Label	Accident	Active	☒
Record Type Name	Accident		
Namespace Prefix			
Description			
Created By	Dhanasri G.	Modified By	Dhanasri G.

8. Create an AutoQuoting Screen Flow:

The screenshot displays the Salesforce Flow Builder interface for a flow named "AutoQuotingFlow - V1". The flow is in the "Developer Edition" and is currently "Inactive". The flow steps are as follows:

- Get Vehicle RT ID (Get Records)
- New Auto Policy Quote - Basic Information (Screen)
- Vehicle_Specific_Details (Screen)
- Create Draft Policy (Create Records)
- End

The "Create Records" step is expanded, showing the following fields and values:

- Field: Customer X, Value: [a] ...basic information > Policy Holder > Record ID X
- Field: Model Year X, Value: [a] Vehicle_Specific_Details > Model year X
- Field: Policy Start Date X, Value: [a] ...y Quote - Basic Information > Policy Start Date X
- Field: Policy State X, Value: [a] ...Policy Quote - Basic Information > Policy State X
- Field: Record Type ID X, Value: [a] ... Type from Get Vehicle RT ID > Record Type ID X
- Field: VIN X, Value: [a] Vehicle_Specific_Details > VIN X

The "Manually assign variables (advanced)" checkbox is checked. The "Store Policy ID in Variable" section is visible, with a search for variables.

9. Create Validation Rule on Policy Object:

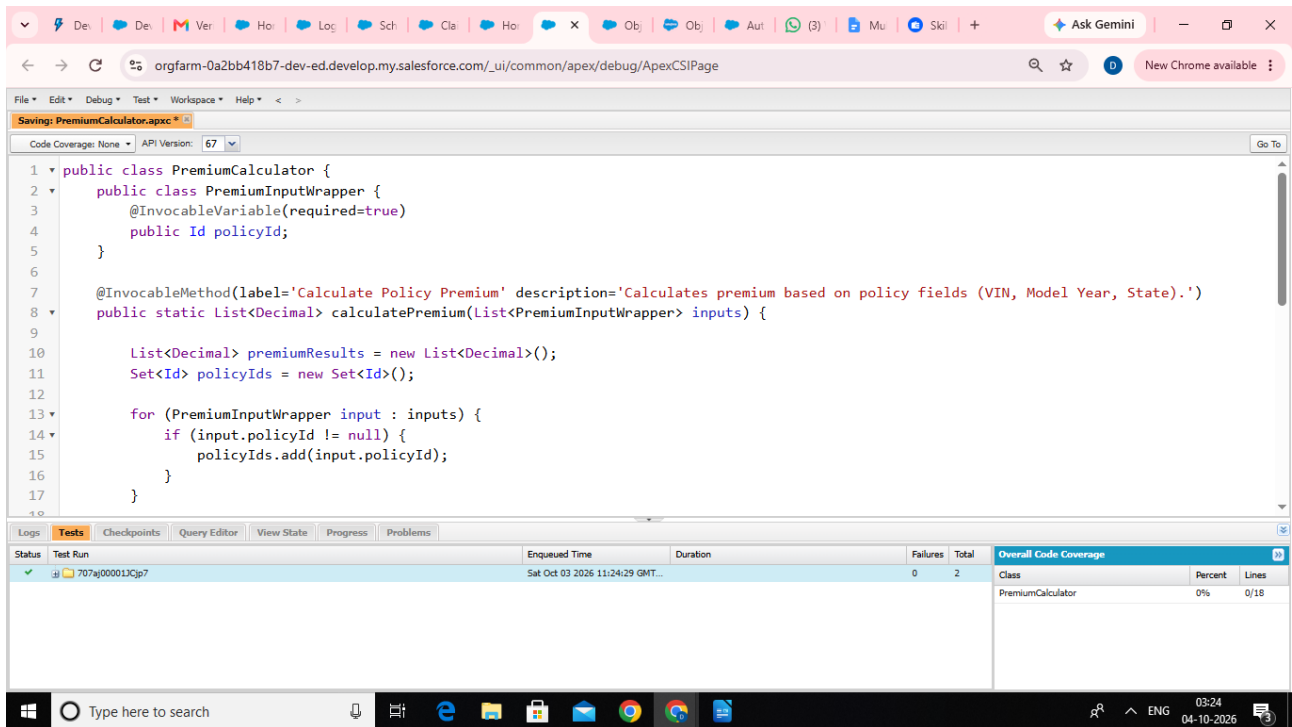
The screenshot displays the Salesforce Object Manager interface for the "Policy" object. The "Validation Rule" section is active, showing the following details:

- Rule Name: VIN_Must_Be_17_Chars
- Rule Condition Formula: AND(RecordType DeveloperName = "Auto", LEN(VIN_c) <> 17)
- Error Message: The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
- Error Location: VIN
- Description: [Blank]
- Created By: [Blank]
- Modified By: [Blank]

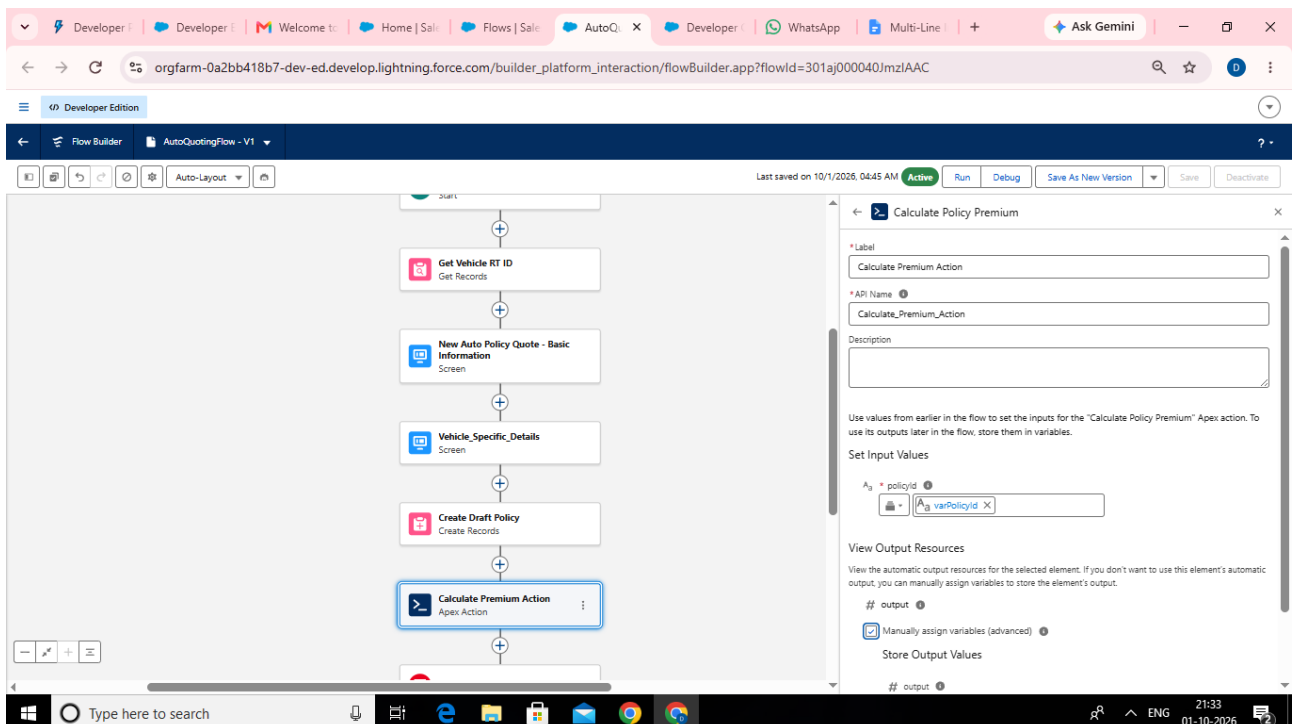
The "Active" checkbox is checked. The "Edit" and "Clone" buttons are visible.

Milestone 2: Complex Policy Issuance & Claim Routing Automation

1. Create an Apex Class (PremiumCalculator) with a method that accepts policy data



2. Integrate this Apex action into the final step of the AutoQuotingFlow to update the draft Policy record



The screenshot displays the Salesforce Flow Builder interface. The flow is titled "AutoQuotingFlow - V1" and is in the "Developer Edition". The flow steps are as follows:

- Get Vehicle RT ID (Get Records)
- New Auto Policy Quote - Basic Information Screen
- Vehicle Specific Details Screen
- Create Draft Policy (Create Records)
- Calculate Premium Action (Apex Action)
- Update Policy With Premium (Update Records)
- End

The "Update Policy With Premium" step is expanded, showing the following configuration:

- Label: Update Policy With Premium
- API Name: Update_Policy_With_Premium
- Description: (Empty)
- How to Find Records to Update and Set Their Values: ☒ Specify conditions to identify records, and set fields individually
- Update Records of This Object Type: Policy
- Filter Policy Records: Condition Requirements to Update Records: All Conditions Are Met (AND)
- Field: Record ID, Operator: Equals, Value: varPolicyId
- Set Field Values for the Policy Records: Field: Premium, Value: varCalculatedPremium

3. Create a Record-Triggered Flow (After Save) on the Claim object

The screenshot shows the "New Automation" dialog box in the Salesforce Flow Builder. The dialog is titled "New Automation" and contains the following sections:

- Get Started with Automations:** Select a category, flow type, use search, or let Agentforce build an automation for you. Includes a search bar and a "Get Started (Beta)" button.
- Categories:** Four categories are listed:
 - Triggered Automations:** Automations launched by records and events. This type of automation runs without user interaction. (View All Automations)
 - Scheduled Automations:** Time-based automations that launch at a specific time or frequency. This type of automation runs without user interaction. (View All Automations)
 - Screen Automations:** Interface-driven automations that guide users through business processes. This type of automation collects or displays... (View All Automations)
 - Autolaunched Automations:** Automations that automatically launch when invoked by APIs, templates, processes, conditions, or something else... (View All Automations)
- Frequently Used:** Four frequently used automation types are listed:
 - Screen Flow:** Guides users through a business process that's launched from Lightning pages. Experience Cloud sites, quick actions, an... (View All Automations)
 - Record-Triggered Flow:** Launches when a record is created, updated, or deleted. This autolaunched flow runs in the background. (View All Automations)
 - Schedule-Triggered Flow:** Launches at a specified time and frequency for each record in a batch. This autolaunched flow runs in the... (View All Automations)
 - Autolaunched Flow (No Trigger):** Launches when invoked by Apex, processes, REST API, and more. This autolaunched flow runs in the... (View All Automations)

Developer Edition

Flow Builder

Record-Triggered Flow Start

Object: Claim

Trigger: A record is created

Optimize for: Actions and Related Records

+ Add Scheduled Paths (Optional)

Open Flow Trigger Explorer for Claim

Run Immediately

+ Get_Policy_RT Get Records

+ Route by Policy Type Decision

Auto Claim Route

Property Claim Route

Life Claim Route

Assign to Auto Queue

Get Records

* Label: Get_Policy_RT

* API Name: Get_Policy_RT

Description:

Get Records of This Object

* Object: Policy

Filter Policy Records

Condition Requirements: All Conditions Are Met (AND)

Field: Record ID Operator: Equals Value: Triggering Claim_c > Policy

+ Add Condition

Sort Policy Records

Sort Order: Ascending

Developer Edition

Flow Builder

+ Add Scheduled Paths (Optional)

Open Flow Trigger Explorer for Claim

Run Immediately

+ Get_Policy_RT Get Records

+ Route by Policy Type Decision

Auto Claim Route

Property Claim Route

Life Claim Route

Assign to Auto Queue Update Records

Decision

* Label: Route by Policy Type

* API Name: Route_by_Policy_Type

Description:

Outcomes

For each path the automation can take, create an outcome. For each outcome, specify the conditions that must be met for the automation to take that path.

OUTCOME ORDER: +

OUTCOME DETAILS

Auto Claim Route

* Outcome Label: Auto Claim Route

* Outcome API Name: Auto_Claim_Route

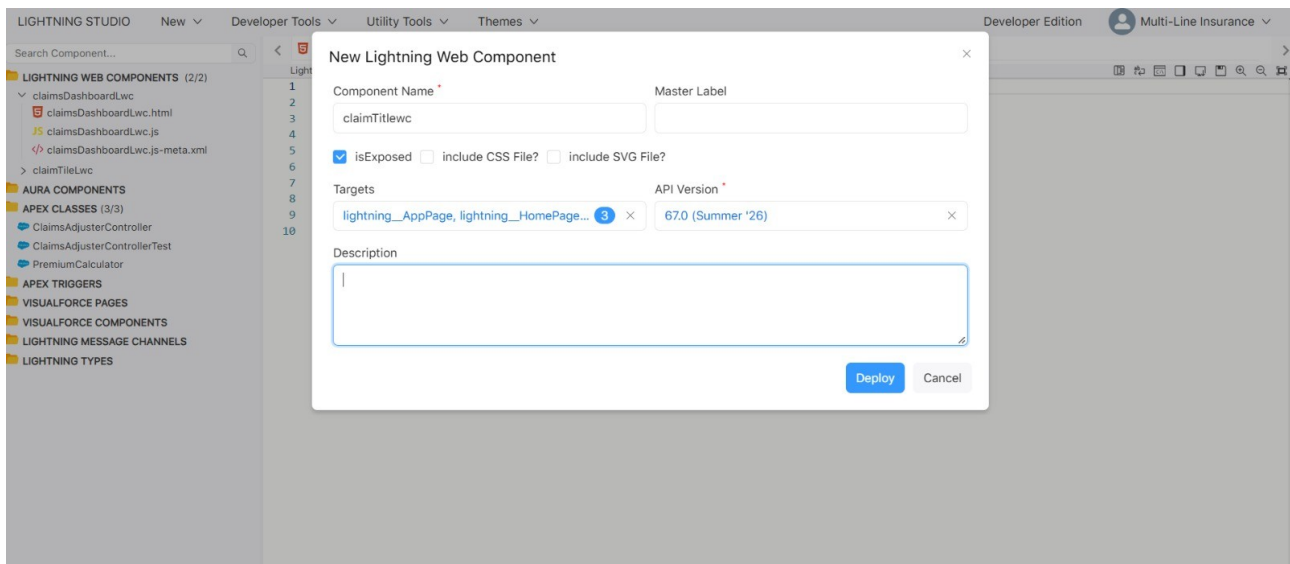
Condition Requirements to Execute Outcome: All Conditions Are Met (AND)

* Resource: varPolicyRTName Operator: Equals Value: Auto

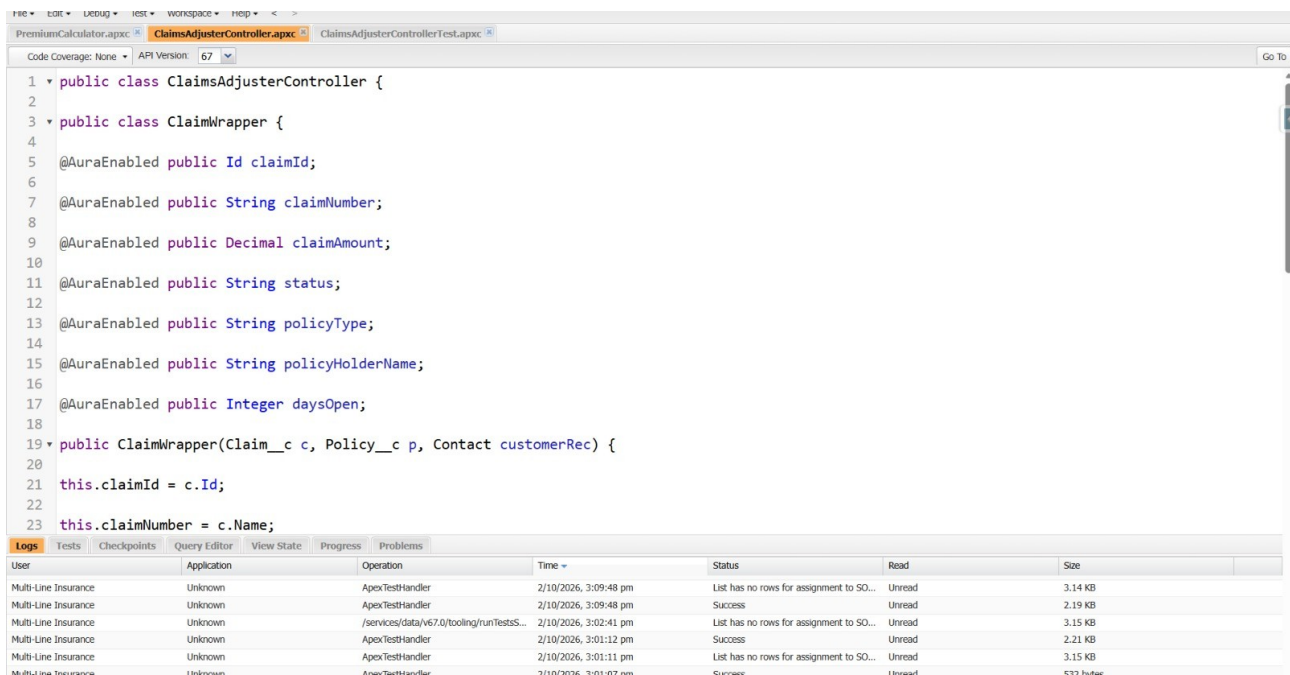
+ Add Condition

Milestone 3: Claims Adjuster LWC Dashboard Development

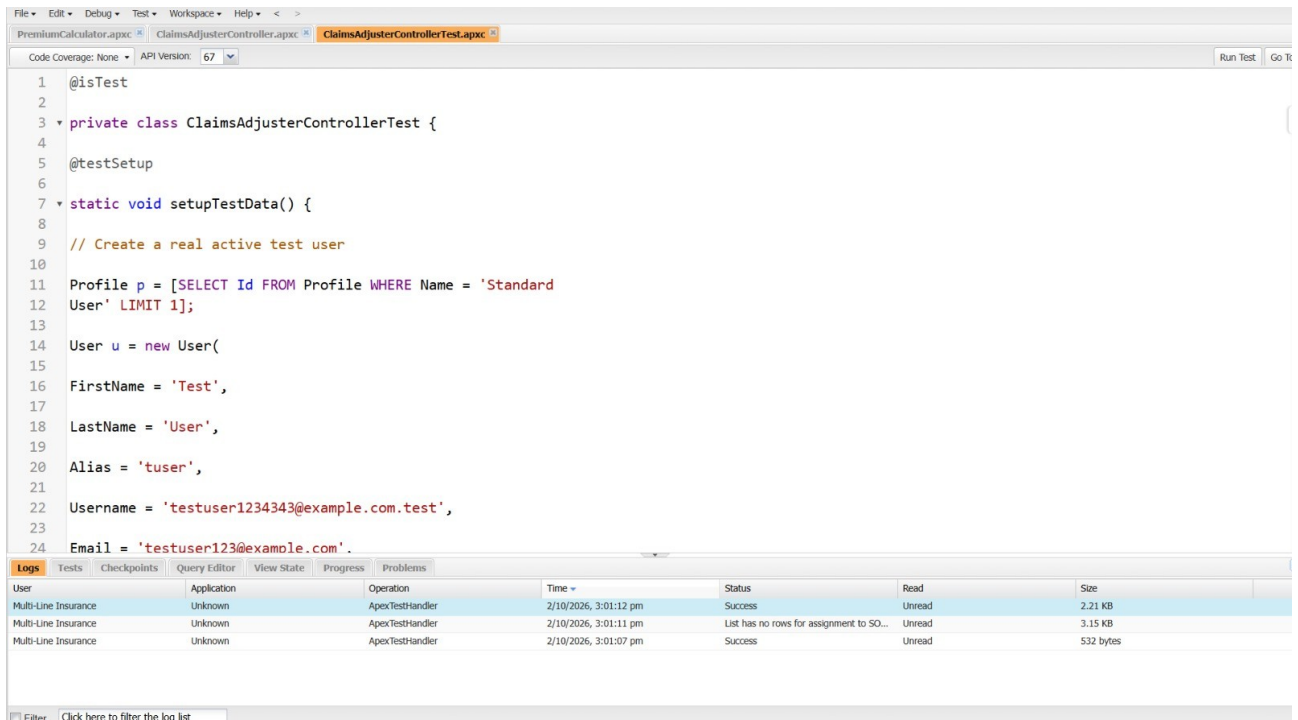
1. Create a robust Apex class (ClaimsAdjusterController) with a method using SOQL to query all Claims owned by the logged-in user, joining related Policy and Customer data. The method must return a custom Wrapper Class listing key metrics.



2. Create a component (claimsDashboardLwc) that uses the Apex method via the @wire service and implements client-side data filtering



3. Create a reusable component (claimTileLwc) to display essential information for each claim



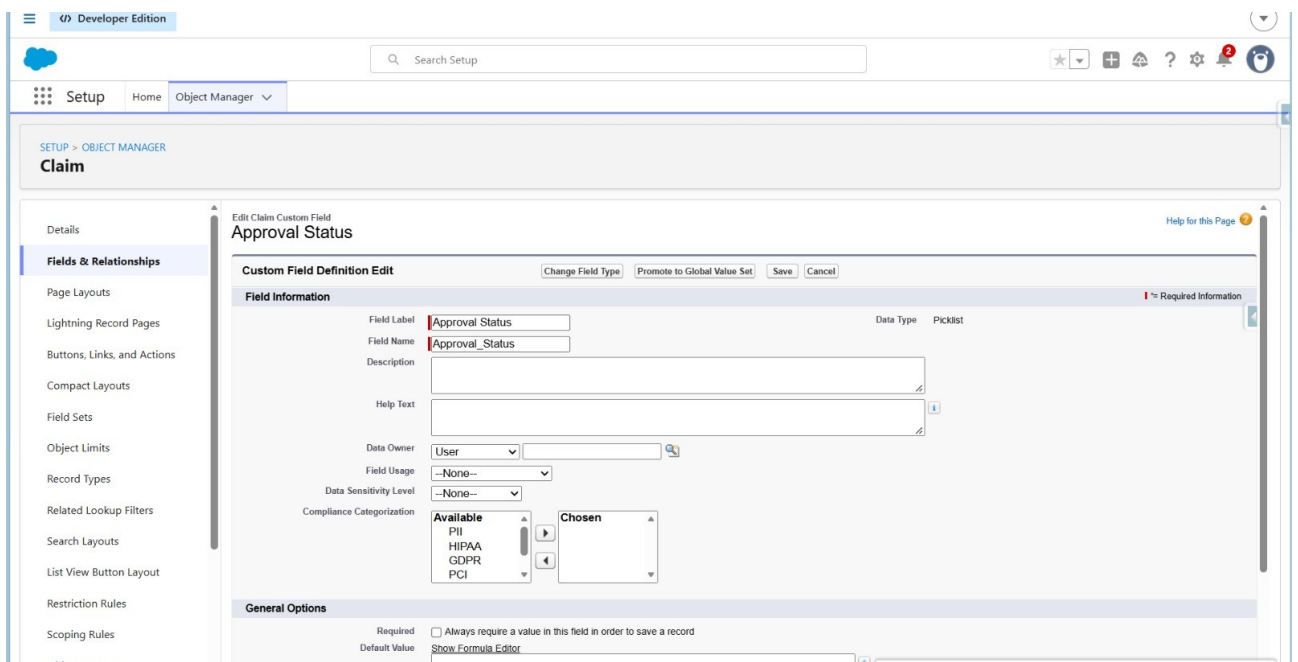
The screenshot shows an IDE with an Apex test class `ClaimsAdjusterControllerTest` and a log table below it.

```
1 @isTest
2
3 private class ClaimsAdjusterControllerTest {
4
5     @testSetup
6
7     static void setupTestData() {
8         // Create a real active test user
9
10
11     Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard
12     User' LIMIT 1];
13
14     User u = new User(
15
16         FirstName = 'Test',
17
18         LastName = 'User',
19
20         Alias = 'tuser',
21
22         Username = 'testuser123434@example.com.test',
23
24         Email = 'testuser123@example.com'.
```

User	Application	Operation	Time	Status	Read	Size
Multi-Line Insurance	Unknown	ApexTestHandler	2/10/2026, 3:01:12 pm	Success	Unread	2.21 KB
Multi-Line Insurance	Unknown	ApexTestHandler	2/10/2026, 3:01:11 pm	List has no rows for assignment to SO...	Unread	3.15 KB
Multi-Line Insurance	Unknown	ApexTestHandler	2/10/2026, 3:01:07 pm	Success	Unread	532 bytes

Milestone 4: Advanced Claim Processing, Security & Code Setup

1. Modify the Claim object to include an Approval Status field.



The screenshot shows the Salesforce Setup interface for defining a custom field named 'Approval Status' on the 'Claim' object.

Field Information

- Field Label: Approval Status
- Field Name: Approval_Status
- Data Type: Picklist

Compliance Categorization

Available	Chosen
PII	
HIPAA	
GDPR	
PCI	

General Options

- Required: ☐ Always require a value in this field in order to save a record
- Default Value: [Show Formula Editor](#)

2. Configure the Approval Process to route first to the Senior Adjuster, then to the Department Manager, using two sequential steps.

The screenshot shows the Salesforce Setup page for Approval Processes. The left sidebar contains navigation links for Data, Feature Settings, and Process Automation. The main content area displays the configuration for the 'Claim: High Value Claim Approval' process.

Process Definition Detail

Field	Value
Process Name	High Value Claim Approval
Unique Name	High_Value_Claim_Approval
Description	
Entry Criteria	Claim: Claim Amount GREATER THAN 50000
Record Editability	Administrator ONLY
Approval Assignment Email Template	Claim Owner
Initial Submitters	Multi-Line Insurance, 10/1/2026, 10:30 AM
Created By	Multi-Line Insurance, 10/1/2026, 10:30 AM
Modified By	Multi-Line Insurance, 10/1/2026, 10:43 AM

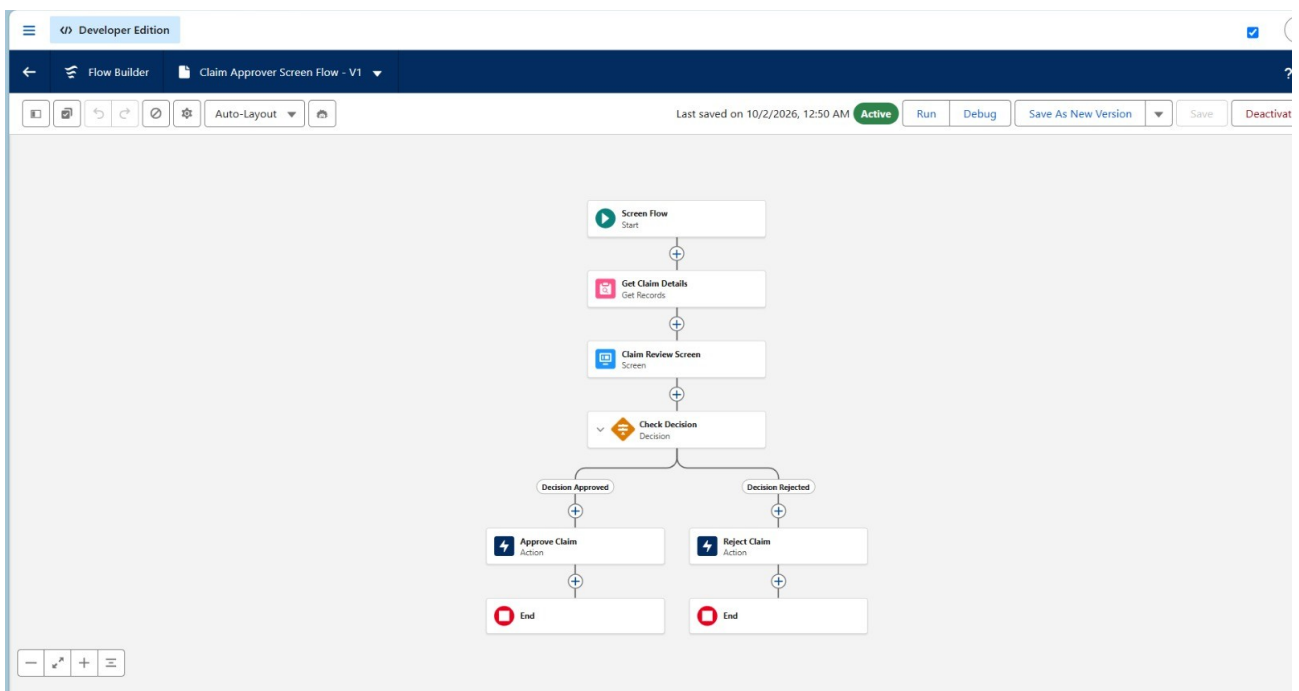
Initial Submission Actions

Action	Type	Description
Record Lock	Record Lock	Lock the record from being edited
Field Update	Field Update	Update Status Submitted

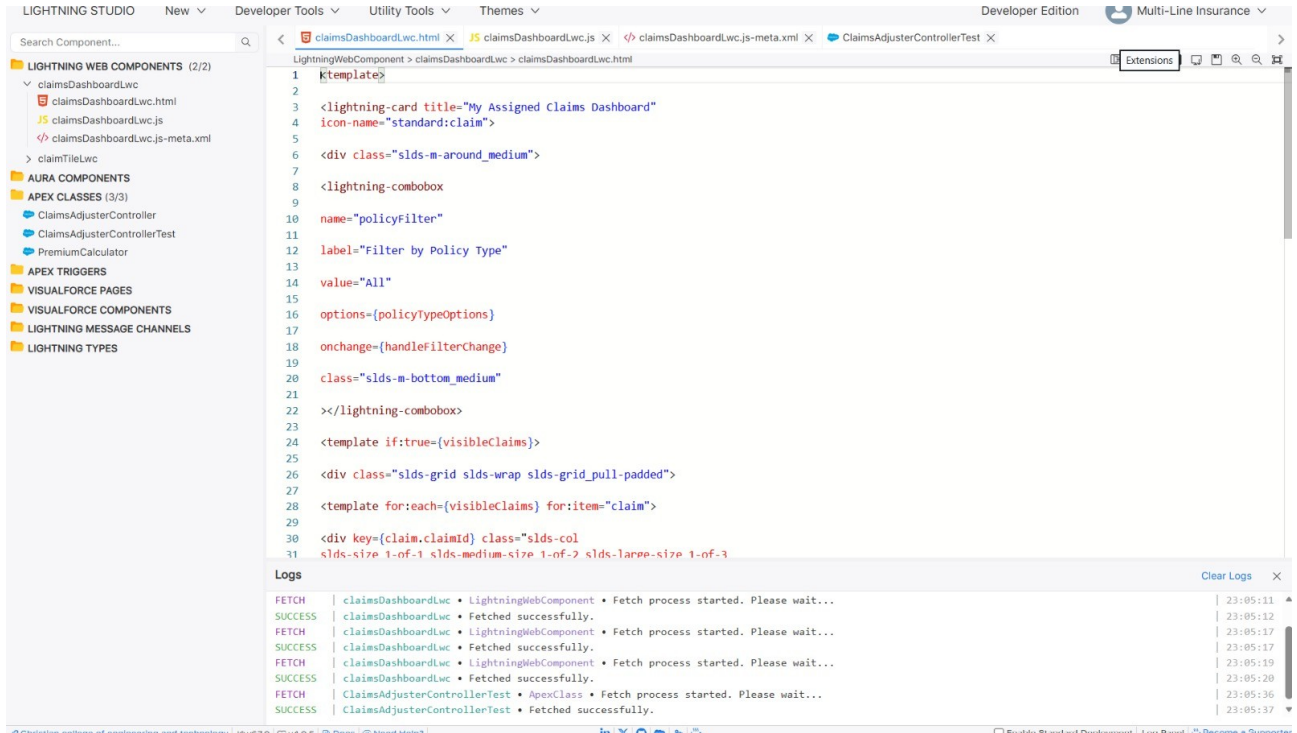
Approval Steps

Action	Step Number	Name	Description	Criteria	Assigned Approver	Reject Behavior
Show Actions Edit	1	Step1 Senior Adjuster			User:Senior Adjuster	Final Rejection
Show Actions Edit	2	Step2 Manager Review			User:Department Manager	Final Rejection

3. Implement a Screen Flow for the approvers to quickly review details and approve/reject directly from the Claim record.



4. Write an extensive Apex Test Class for the ClaimsAdjusterController with 95%+ code coverage. Include test methods for complex claim filtering and data retrieval logic.

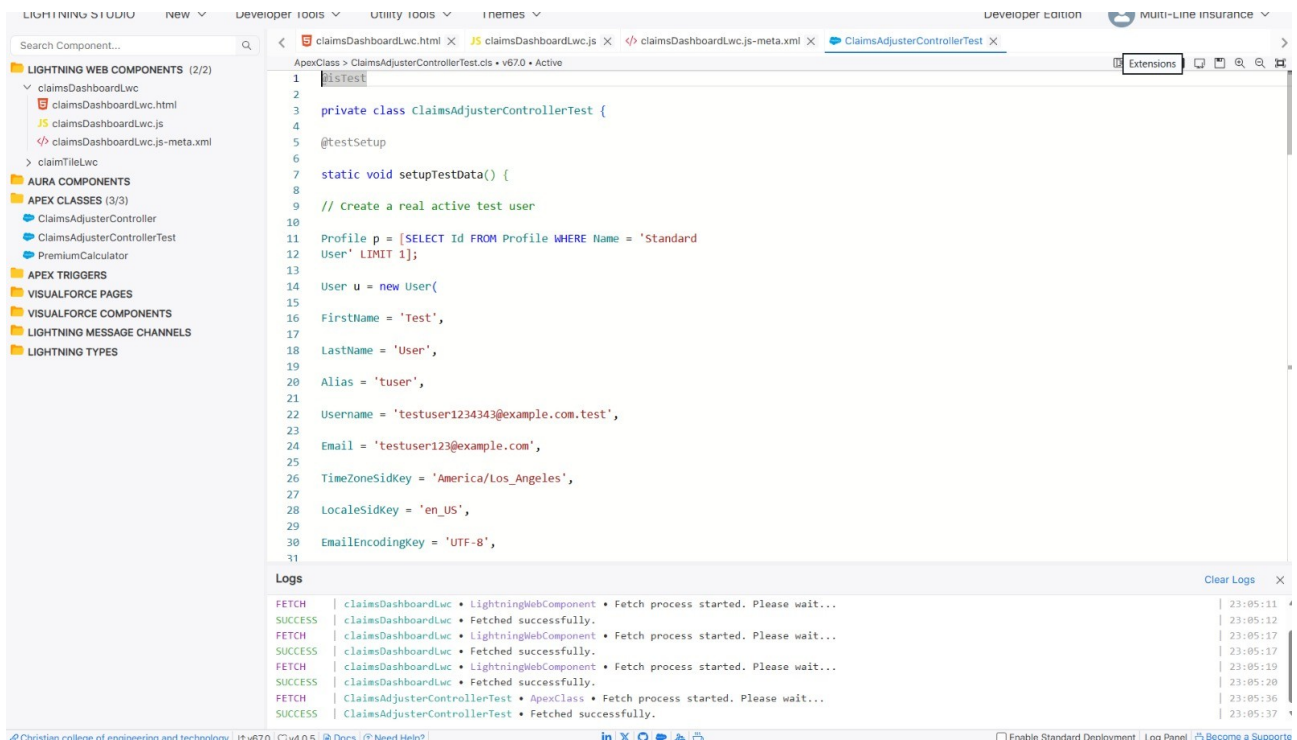


The screenshot shows the Lightning Studio interface with the Apex test class `ClaimsAdjusterControllerTest` open. The class is a private class with a `@testSetup` method and a `static void setupTestData()` method. The `setupTestData()` method creates a test user with the following attributes:

```
1 @testSetup
2
3 private class ClaimsAdjusterControllerTest {
4
5     static void setupTestData() {
6
7         // Create a real active test user
8
9         Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard
10         User' LIMIT 1];
11
12         User u = new User(
13
14             FirstName = 'Test',
15
16             LastName = 'User',
17
18             Alias = 'tuser',
19
20             Username = 'testuser123434@example.com.test',
21
22             Email = 'testuser123@example.com',
23
24             TimeZoneSidKey = 'America/Los_Angeles',
25
26             LocaleSidKey = 'en_US',
27
28             EmailEncodingKey = 'UTF-8',
29
30         );
31     }
32 }
```

The logs show the following messages:

```
1 Fetch process started. Please wait...
2 Fetch process started. Please wait...
3 Fetch process started. Please wait...
4 Fetch process started. Please wait...
5 Fetch process started. Please wait...
6 Fetch process started. Please wait...
7 Fetch process started. Please wait...
8 Fetch process started. Please wait...
9 Fetch process started. Please wait...
10 Fetch process started. Please wait...
```

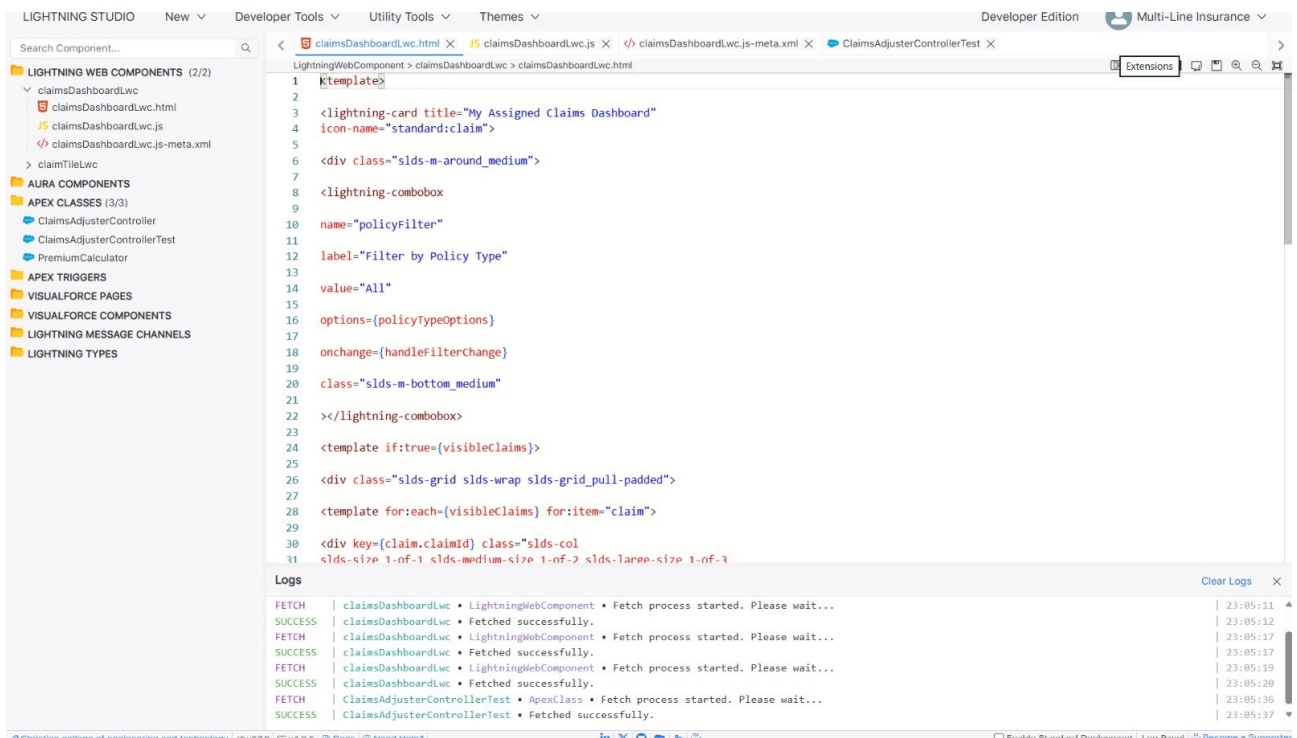


The screenshot shows the Lightning Studio interface with the Apex test class `ClaimsAdjusterControllerTest` open. The class is a private class with a `@testSetup` method and a `static void setupTestData()` method. The `setupTestData()` method creates a test user with the following attributes:

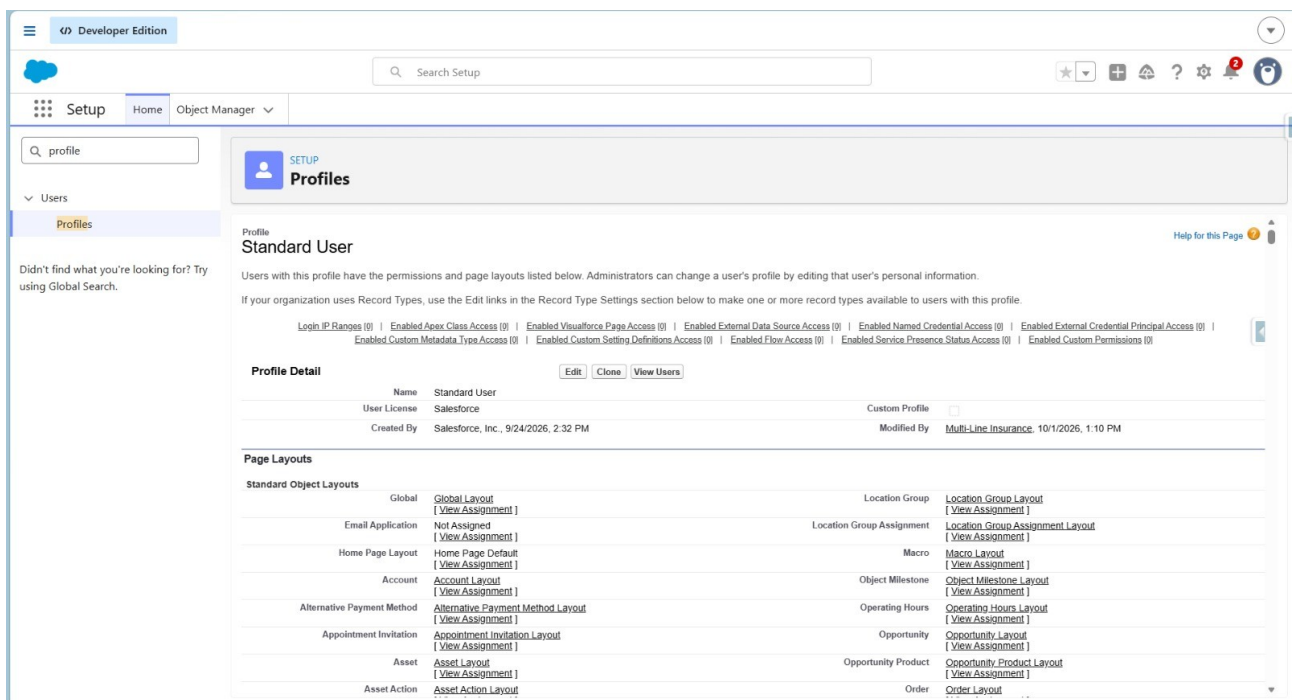
```
1 @testSetup
2
3 private class ClaimsAdjusterControllerTest {
4
5     static void setupTestData() {
6
7         // Create a real active test user
8
9         Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard
10         User' LIMIT 1];
11
12         User u = new User(
13
14             FirstName = 'Test',
15
16             LastName = 'User',
17
18             Alias = 'tuser',
19
20             Username = 'testuser123434@example.com.test',
21
22             Email = 'testuser123@example.com',
23
24             TimeZoneSidKey = 'America/Los_Angeles',
25
26             LocaleSidKey = 'en_US',
27
28             EmailEncodingKey = 'UTF-8',
29
30         );
31     }
32 }
```

The logs show the following messages:

```
1 Fetch process started. Please wait...
2 Fetch process started. Please wait...
3 Fetch process started. Please wait...
4 Fetch process started. Please wait...
5 Fetch process started. Please wait...
6 Fetch process started. Please wait...
7 Fetch process started. Please wait...
8 Fetch process started. Please wait...
9 Fetch process started. Please wait...
10 Fetch process started. Please wait...
```

5. Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state.



Developer Edition

Search Setup

Setup Home Object Manager

profile

Users

Profiles

Didn't find what you're looking for? Try using Global Search.

Profiles

Department Manager Profile

Users with this profile have the permissions and page layouts listed below. Administrators can change a user's profile by editing that user's personal information.

If your organization uses Record Types, use the Edit links in the Record Type Settings section below to make one or more record types available to users with this profile.

Login IP Ranges (0) | Enabled Apex Class Access (0) | Enabled Visualforce Page Access (0) | Enabled External Data Source Access (0) | Enabled Named Credential Access (0) | Enabled External Credential Principal Access (0) | Enabled Custom Metadata Type Access (0) | Enabled Custom Setting Definitions Access (0) | Enabled Flow Access (0) | Enabled Service Presence Status Access (0) | Enabled Custom Permissions (0)

Profile Detail

Edit Clone Delete View Users

Name	Department Manager Profile
User License	Salesforce
Description	Custom Profile
Created By	Multi-Line Insurance, 10/1/2026, 10:01 AM
Modified By	Multi-Line Insurance, 10/3/2026, 9:57 AM

Page Layouts

Standard Object Layouts		
Global	Global Layout [View Assignment]	Location Group
Email Application	Not Assigned [View Assignment]	Location Group Assignment
Home Page Layout	Home Page Default [View Assignment]	Macro
Account	Account Layout [View Assignment]	Object Milestone
Alternative Payment Method	Alternative Payment Method Layout [View Assignment]	Operating Hours
Appointment Invitation	Appointment Invitation Layout [View Assignment]	Opportunity
Asset	Asset Layout [View Assignment]	Opportunity Product

6. Finalize all Permission Sets to differentiate access between Agents (Quoting only), Adjusters (Claim handling), and Managers (Reporting/Approvals).

Developer Edition

Search Setup

Setup Home Object Manager

User

Users

Users

Senior Adjuster

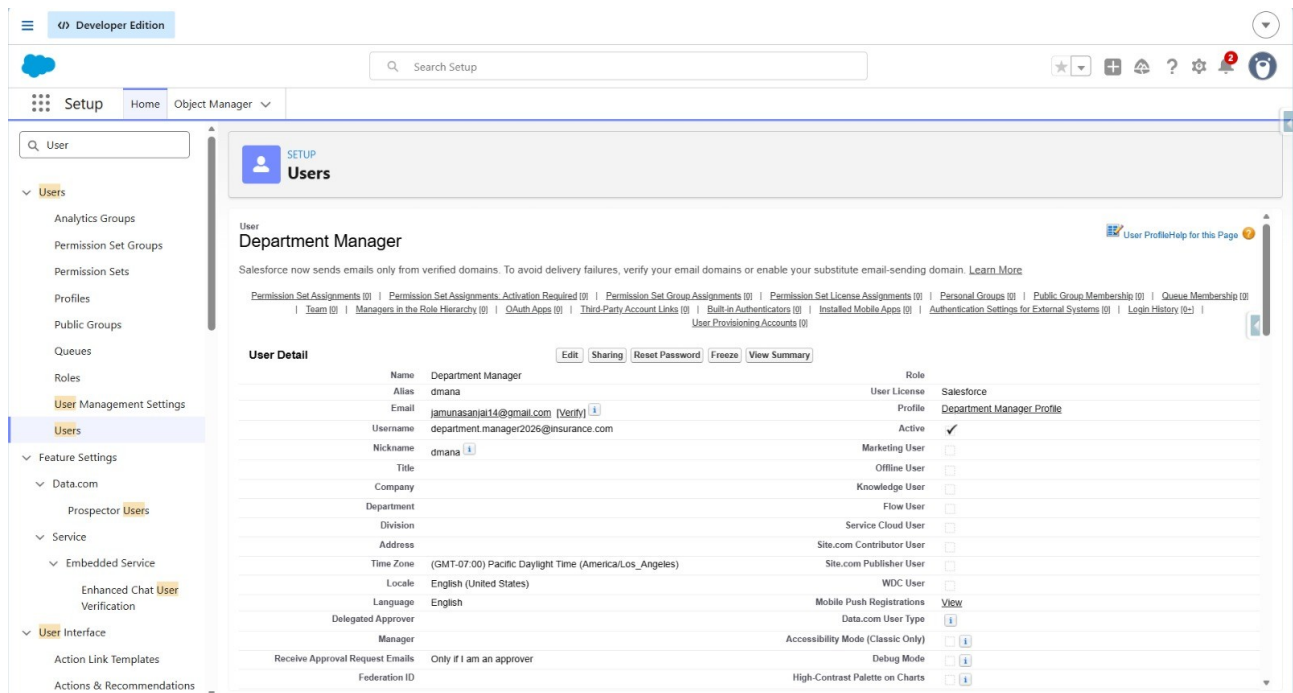
Salesforce now sends emails only from verified domains. To avoid delivery failures, verify your email domains or enable your substitute email-sending domain. [Learn More](#)

Permission Set Assignments (0) | Permission Set Assignments: Activation Required (0) | Permission Set Group Assignments (0) | Permission Set License Assignments (0) | Personal Groups (0) | Public Group Membership (0) | Queue Membership (0) | Team (0) | Managers in the Role Hierarchy (0) | OAuth Apps (0) | Third-Party Account Links (0) | Built-in Authenticators (0) | Installed Mobile Apps (0) | Authentication Settings for External Systems (0) | Login History (0) | User Provisioning Accounts (0)

User Detail

Edit Sharing Reset Password Freeze View Summary

Name	Senior Adjuster	Role	Salesforce
Alias	sadju	User License	Salesforce
Email	jamunasanjat14@gmail.com [Verify]	Profile	Senior Adjuster Profile
Username	senior.adjuster2026@insurance.com	Active	☒
Nickname	sadju	Marketing User	☐
Title		Offline User	☐
Company		Knowledge User	☐
Department		Flow User	☐
Division		Service Cloud User	☐
Address		Site.com Contributor User	☐
Time Zone	(GMT-07:00) Pacific Daylight Time (America/Los_Angeles)	Site.com Publisher User	☐
Locale	English (United States)	WDC User	☐
Language	English	Mobile Push Registrations	View
Delegated Approver		Data.com User Type	1
Manager		Accessibility Mode (Classic Only)	☐ 1
Receive Approval Request Emails	Only if I am an approver	Debug Mode	☐ 1
Federation ID		High-Contrast Palette on Charts	☐ 1



Demo Link:

https://drive.google.com/file/d/1ohWsA4CVy80yDtJa29wW20R8b3ivLC_C/view?usp=drivesdk

GitHub Link:

[https://github.com/dhanasri1510/Multi-Line Insurance Policy and Claim](https://github.com/dhanasri1510/Multi-Line_Insurance_Policy_and_Claim)

7.FUNCTIONAL & PERFORMANCE TESTING

The Multi-Line Insurance Policy and Claims Management System was tested to check whether the main features work correctly.

- Customer creation and customer information updates were tested.
- Insurance product creation and management were tested.
- Insurance policy creation and status updates were tested.
- Policy renewal and expiry-related processes were verified.
- Claim creation and claim status updates were tested.
- Premium payment information and payment status were checked.
- Salesforce Flows and automated processes were verified.
- Data retrieval and user permissions were checked.
- AgentForce AI responses for policy and claim-related queries were tested.
- Reports and dashboards were verified.

8. ADVANTAGES

- Centralized customer and insurance information.
- Supports multiple insurance lines such as Life, Health, Motor and Property Insurance.
- Easy insurance policy management.
- Easy claim registration and status tracking.
- Reduces manual work through automation.
- Provides quick access to policy and claim information.
- Automated policy renewal and expiry reminders.
- AI-based assistance through AgentForce.
- Reports and dashboards help monitor policies and claims.

DISADVANTAGES

- Requires accurate customer, policy and claim data.
- Depends on proper Salesforce configuration.
- AI responses depend on available data and properly configured prompts.
- Requires appropriate user permissions and security settings.
- System maintenance and configuration updates may be required when business requirements change.

9. FUTURE SCOPE

The future improvements of the Multi-Line Insurance Policy and Claims Management System include:

- Enhance AgentForce AI for more insurance-related queries.
- Improve automated claim processing.
- Add AI-based claim fraud detection.
- Provide customer self-service through Experience Cloud.
- Add online policy renewal and premium payment facilities.

10. CONCLUSION

The Multi-Line Insurance Policy and Claims Management System provides a centralized Salesforce-based solution for managing different types of insurance policies and claims. It uses Salesforce Objects, Flows, AgentForce AI, Prompt Builder, Reports and Dashboards to manage customer, insurance product, policy, claim and payment information efficiently.

The system helps improve policy management, simplify claim tracking, reduce manual work and provide timely insurance-related information to customers and insurance staff. The use of Salesforce automation and AgentForce AI also provides a foundation for future improvements in insurance management and customer assistance.